🎯 Lecture 21: Practice Problems — Technical Case Studies

Jupyter questions.ipynb

PDF questions.pdf

Solutions View Solutions

🌐 **Language: English** | हिंदी / Hinglish | View Complete Solutions | Overview

---

📌 Linear Algebra Practice Suite: Overview & Instructions

Linear Algebra is the foundational mathematical backbone of modern Artificial Intelligence and Deep Learning. From spatial coordinate transformations in Computer Vision to separating hyperplanes in Support Vector Machines, and spectral dimensionality reduction in high-dimensional financial factor modeling, AI algorithms rely on vectors, matrices, determinants, and eigendecomposition.

This practice module provides **3 comprehensive industry-grade case studies**:

1. **Case 1:** Computer Vision 2D Affine Transformations & Homogeneous Coordinates (Robotics & PyTorch Augmentation Pipelines)
2. **Case 2:** Support Vector Machine (SVM) Maximum-Margin Hyperplane & Vector Projections (Machine Learning Geometric Optimization)
3. **Case 3:** Quantitative Finance Risk & Sensor PCA Eigendecomposition (Multi-Asset Factor Risk & Spectral Analysis)

📌 Case Study 1: Computer Vision 2D Affine Transformations & Homogeneous Coordinates

🌐 Real-World Context & Engineering Problem

In modern computer vision and autonomous robotics pipelines (such as PyTorch `torchvision.transforms`, OpenCV, and Spatial Transformer Networks), image geometric transformations are modeled via linear maps in 2D projective space $\mathbb{P}^2$.

In standard 2D Cartesian coordinates (x, y), **Rotation** and **Shear** are linear operations, but **Translation** (coordinate shift) is non-linear:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \end{bmatrix}$$

This translation cannot be expressed as a 2×2 matrix multiplication. To resolve this, computer vision engineers embed 2D coordinates into **Homogeneous Coordinates** $\tilde{\mathbf{x}} = [x, y, 1]^T$, converting non-linear translation into an elegant linear 3×3 matrix multiplication.

An autonomous inspection robot camera captures a rectangular object whose polygon vertices are represented by the matrix $V \in \mathbb{R}^{3\times 4}$:

$$V = \begin{bmatrix} 0 & 2 & 2 & 0 \\ 0 & 0 & 3 & 3 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

The 4 columns represent 4 vertices:

- Vertex 1: $(0, 0)$
- Vertex 2: $(2, 0)$
- Vertex 3: $(2, 3)$
- Vertex 4: $(0, 3)$

The initial rectangle has width 2 units and height 3 units (Initial Area $= 6.0$ square units).

The vision augmentation pipeline requires applying **3 sequential geometric operations**:

1. **Rotation:** Counter-clockwise rotation around the origin $(0, 0)$ by angle $\theta = 45^\circ = \frac{\pi}{4}$.
2. **Horizontal Shear:** Horizontal shearing with shear factor $k_x = 0.5$.
3. **Translation:** Coordinate displacement by offset vector $\mathbf{t} = [3, -2]^T$ ($t_x = 3.0, t_y = -2.0$).

📋 Questions Asked in This Case Study (Clear Objectives in Text Form)

You are required to address and solve the following **5 specific engineering deliverables**:

- **Bullet Point 1:** Construct explicit 3×3 Homogeneous Matrices for the three elementary transformations: Rotation $R(\theta)$, Horizontal Shear $Sh(k_x)$, and Translation $T(t_x, t_y)$.
- **Bullet Point 2:** Compute the composite transformation matrix $M_1 = T \cdot Sh \cdot R$ and the alternate sequence $M_2 = R \cdot T \cdot Sh$. Prove mathematically that matrix multiplication

does not commute ($M_1 \neq M_2$) and explain why transformation ordering is critical in computer vision pipelines.
- **Bullet Point 3:** Calculate the determinant of the upper-left 2×2 linear block of M_1. Prove that translation does not alter the area of the polygon, and evaluate the final Area Scaling Factor.
- **Bullet Point 4:** Explain why Forward Warping generates rasterization holes (unmapped destination pixels) in digital image processing, and derive the explicit analytical inverse matrix M_1^{-1} required for Backward Warping.
- **Bullet Point 5:** Implement an end-to-end Python script using NumPy to compute transformed vertex coordinates and visualize both original and transformed polygons on a coordinate grid.

🔍 Detailed In-Depth Explanation of Each Bullet Point

1. Bullet Point 1 Explanation: Elementary Homogeneous Matrices Construction

- **What It Means:** Standard 2×2 matrices cannot accommodate translation because translation shifts the origin. Homogeneous coordinates append a dummy scale factor $w = 1$ to form $[x, y, 1]^T$.
- **Why It Is Essential:** Standardizing all operations into 3×3 matrices allows hardware accelerators (GPUs) to chain multiple complex geometric transformations into a single matrix product.
- **How to Formulate:** Construct $R(\pi/4)$ using $\cos(\pi/4) = \sin(\pi/4) = \frac{\sqrt{2}}{2}$, horizontal shear $Sh(0.5)$, and translation $T(3, -2)$ as explicit 3×3 matrices.

2. Bullet Point 2 Explanation: Matrix Composition & Non-Commutativity ($M_1 \neq M_2$)

- **What It Means:** Investigate whether changing the order of transformation operations produces different geometric results.
- **Why It Is Essential:** In vision pipelines, translating an object before rotating it rotates the translation offset itself, displacing the object along an unintended orbital trajectory.
- **How to Formulate:**
  - Standard Pipeline: $M_1 = T \cdot Sh \cdot R$ (Rotate first, shear second, translate last).
  - Alternate Pipeline: $M_2 = R \cdot T \cdot Sh$.
  - Evaluate both products and demonstrate why $M_1 \neq M_2$.

3. Bullet Point 3 Explanation: Determinant & Area Scaling Factor

- **What It Means:** The determinant of a transformation matrix measures the volumetric or areal scaling induced by the linear map.
- **Why It Is Essential:** Evaluating whether data augmentation alters the physical scale/area of detected bounding boxes and segmentation masks.

- **How to Formulate:** Compute $\det(A) = \det(Sh) \cdot \det(R)$ for the 2×2 block. Show that $|\det(A)| = 1.0$, proving that rotation and shear preserve area exactly, and translation has no effect on area.

4. Bullet Point 4 Explanation: Backward Warping & Inverse Matrix M_1^{-1}

- **What It Means:** In forward mapping, rounding floating-point coordinates causes missing pixels in the output image. Backward warping traverses destination grid points and queries their source coordinates via M_1^{-1}.
- **Why It Is Essential:** Backward warping enables bilinear and bicubic interpolation without gaps.
- **How to Formulate:** Apply the reversal law of matrix inverses: $M_1^{-1} = (T \cdot Sh \cdot R)^{-1} = R^{-1} \cdot Sh^{-1} \cdot T^{-1}$ and calculate explicit numerical components.

5. Bullet Point 5 Explanation: Python Pipeline & Visual Grid

- **What It Means:** Convert mathematical formulations into executable NumPy code and Matplotlib plots.
- **Why It Is Essential:** Real-world engineering requires code verification and visual sanity checks.
- **How to Formulate:** Compute $V' = M_1 V$, print transformed coordinates, and plot polygons side-by-side.

🎯 Formal Mathematical Equations

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad Sh(k_x) = \begin{bmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

📌 Case Study 2: Support Vector Machine (SVM) Maximum-Margin Hyperplane & Vector Projections

🏢 Real-World Context & Engineering Problem

Support Vector Machines (SVM) find the optimal linear separating hyperplane in feature space that maximizes the geometric distance (margin γ) to the closest training examples of any class.

In a 2D feature space $\mathbb{R}^2$, consider a linearly separable dataset:

- **Class $+1$ Samples:** $\mathbf{x}_1 = [1, 2]^T, \mathbf{x}_2 = [1, 3]^T$
- **Class -1 Samples:** $\mathbf{x}_3 = [3, 1]^T, \mathbf{x}_4 = [4, 2]^T$

The decision boundary is defined by parameter vector $\mathbf{w} = [w_1, w_2]^T$ and scalar bias $b \in \mathbb{R}$:

$$H = \{\mathbf{x} \in \mathbb{R}^2 \mid \mathbf{w}^T\mathbf{x} + b = 0\}$$

📋 Questions Asked in This Case Study (Clear Objectives in Text Form)

You are required to address and solve the following **5 specific geometric deliverables**:

- **Bullet Point 1:** Prove from first principles that the parameter weight vector $\mathbf{w}$ is strictly orthogonal (perpendicular) to the separating hyperplane H.
- **Bullet Point 2:** Using vector projection along the unit normal $\hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|_2}$, derive the formula for the orthogonal signed Euclidean distance from any arbitrary point $\mathbf{x}_0$ to the hyperplane:

  $$d(\mathbf{x}_0, H) = \frac{\mathbf{w}^T\mathbf{x}_0 + b}{\|\mathbf{w}\|_2}$$

- **Bullet Point 3:** Prove that if the canonical supporting hyperplanes are defined by $\mathbf{w}^T\mathbf{x} + b = +1$ and $\mathbf{w}^T\mathbf{x} + b = -1$, the total geometric margin between them is $\gamma = \frac{2}{\|\mathbf{w}\|_2}$.
- **Bullet Point 4:** Given active support vectors $\mathbf{x}_1 = [1, 2]^T$ and $\mathbf{x}_3 = [3, 1]^T$, analytically calculate the optimal weight vector $\mathbf{w}^*$, scalar bias b^*, and margin width γ.
- **Bullet Point 5:** Derive the explicit line equation of the decision boundary, verify that all four training points satisfy the canonical margin constraints $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$, and implement Python visualization code.

🔍 Detailed In-Depth Explanation of Each Bullet Point

1. Bullet Point 1 Explanation: Normal Vector Orthogonality Proof

- **What It Means:** Prove that every vector lying inside the hyperplane forms a 90° angle with $\mathbf{w}$.
- **Why It Is Essential:** Establishes the geometric foundation of linear decision boundaries.
- **How to Formulate:** Take two distinct points $\mathbf{x}_a, \mathbf{x}_b \in H$. Show that their displacement vector $(\mathbf{x}_a - \mathbf{x}_b)$ satisfies $\mathbf{w}^T(\mathbf{x}_a - \mathbf{x}_b) = 0$.

2. Bullet Point 2 Explanation: Point-to-Hyperplane Distance Derivation

- **What It Means:** Measure the exact Euclidean perpendicular distance from any point $\mathbf{x}_0$ to the hyperplane.
- **Why It Is Essential:** Needed to compute margin violations and loss functions.
- **How to Formulate:** Project the vector from an arbitrary hyperplane point $\mathbf{x}_p$ to $\mathbf{x}_0$ onto the unit normal $\hat{\mathbf{w}}$.

3. Bullet Point 3 Explanation: Geometric Margin Formula $\gamma = \frac{2}{\|\mathbf{w}\|_2}$

- **What It Means:** Derive the width of the empty margin corridor separating the two classes.
- **Why It Is Essential:** Shows why maximizing geometric margin is mathematically equivalent to minimizing the weight vector norm $\frac{1}{2}\|\mathbf{w}\|_2^2$.
- **How to Formulate:** Compute the distance between parallel hyperplanes $\mathbf{w}^T\mathbf{x} + b = +1$ and $\mathbf{w}^T\mathbf{x} + b = -1$.

4. Bullet Point 4 Explanation: Analytical Parameter Determination

- **What It Means:** Solve the exact system of linear equations to determine numerical values for $w_1, w_2, b,$ and γ.
- **Why It Is Essential:** Provides ground-truth analytical verification before running iterative quadratic programming solvers.
- **How to Formulate:** Set up canonical support vector equations and solve for $\mathbf{w}^* = [-0.8, 0.4]^T$ and $b^* = 1.0$.

5. Bullet Point 5 Explanation: Verification & Visualization

- **What It Means:** Verify margin inequalities $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ across all samples and plot the hyperplane.
- **Why It Is Essential:** Confirms linear separability without margin violations.

📌 Case Study 3: Quantitative Finance Risk & Sensor PCA Eigendecomposition

🏢 Real-World Context & Engineering Problem

A quantitative hedge fund tracks the daily returns of $p = 4$ correlated financial asset classes (Equities, Bonds, Commodities, FX) across $N = 500$ trading days. The empirical covariance matrix of centered returns $X_c \in \mathbb{R}^{500 \times 4}$ is:

$$\boldsymbol{\Sigma} = \begin{bmatrix} 4.0 & 2.0 & 1.0 & 0.0 \\ 2.0 & 3.0 & 0.5 & 0.5 \\ 1.0 & 0.5 & 2.0 & 0.0 \\ 0.0 & 0.5 & 0.0 & 1.0 \end{bmatrix}$$

The diagonal entries represent asset variances, and off-diagonal entries represent cross-asset covariances. Multi-collinearity makes portfolio optimization ill-conditioned. The research team deploys **Principal Component Analysis (PCA)** to find orthogonal synthetic factor portfolios.

📋 Questions Asked in This Case Study (Clear Objectives in Text Form)

You are required to address and solve the following **5 advanced spectral linear algebra deliverables**:

- **Bullet Point 1:** Prove mathematically that any sample covariance matrix $\boldsymbol{\Sigma} = \frac{1}{N-1} X_c^T X_c$ is symmetric and positive semi-definite (PSD). State why all its eigenvalues must be real and non-negative ($\lambda_i \geq 0$).
- **Bullet Point 2:** Using constrained optimization with Lagrange Multipliers, prove from first principles that the unit direction vector $\mathbf{q}_1$ ($\|\mathbf{q}_1\|_2 = 1$) that maximizes variance is the principal eigenvector of $\boldsymbol{\Sigma}$.
- **Bullet Point 3:** Compute all 4 eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \lambda_4$, verify that the sum of eigenvalues equals the matrix trace $\text{Tr}(\boldsymbol{\Sigma}) = 10.0$, and calculate the Proportion of Explained Variance (PEV_k) for each principal component.
- **Bullet Point 4:** Determine the minimum number of principal components required to preserve at least 80% of total portfolio risk/variance.
- **Bullet Point 5:** Construct the rank-2 reconstructed covariance matrix $\boldsymbol{\Sigma}_2 = \sum_{i=1}^{2} \lambda_i \mathbf{q}_i \mathbf{q}_i^T$ and compute the Frobenius reconstruction error $\|\boldsymbol{\Sigma} - \boldsymbol{\Sigma}_2\|_F$.

🔍 Detailed In-Depth Explanation of Each Bullet Point

1. Bullet Point 1 Explanation: Symmetry, PSD & Non-Negative Eigenvalues

- **What It Means:** Prove $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}^T$ and $\mathbf{v}^T \boldsymbol{\Sigma} \mathbf{v} \geq 0$ for all $\mathbf{v} \in \mathbb{R}^p$.
- **Why It Is Essential:** Portfolio variance $\mathbf{v}^T \boldsymbol{\Sigma} \mathbf{v}$ cannot physically be negative. A non-PSD covariance matrix would imply negative risk, breaking financial models.
- **How to Formulate:** Show that $\boldsymbol{\Sigma}^T = (\frac{1}{N-1} X_c^T X_c)^T = \frac{1}{N-1} X_c^T X_c$. Then show $\mathbf{v}^T \boldsymbol{\Sigma} \mathbf{v} = \frac{1}{N-1} \|X_c \mathbf{v}\|_2^2 \geq 0$.

2. Bullet Point 2 Explanation: Lagrange Multipliers Formulation

- **What It Means:** Prove that finding the direction of maximal variance naturally produces the matrix eigenvector equation $\boldsymbol{\Sigma}\mathbf{q} = \lambda\mathbf{q}$.
- **Why It Is Essential:** Unifies multivariate statistics and linear algebra through calculus.
- **How to Formulate:** Formulate Lagrangian $\mathcal{L}(\mathbf{q}_1, \lambda) = \mathbf{q}_1^T\boldsymbol{\Sigma}\mathbf{q}_1 - \lambda(\mathbf{q}_1^T\mathbf{q}_1 - 1)$, compute $\nabla_{\mathbf{q}_1}\mathcal{L} = \mathbf{0}$, and derive $\boldsymbol{\Sigma}\mathbf{q}_1 = \lambda\mathbf{q}_1$.

3. Bullet Point 3 Explanation: Eigendecomposition, Trace & Variance Spectrum

- **What It Means:** Decompose $\boldsymbol{\Sigma}$ into principal spectral factors and compute the variance percentage captured by each factor.
- **Why It Is Essential:** Identifies the latent risk drivers across the portfolio.
- **How to Formulate:** Compute eigenvalues, verify $\sum\lambda_i = \mathrm{Tr}(\boldsymbol{\Sigma}) = 10.0$, and evaluate $\mathrm{PEV}_k = \frac{\lambda_k}{\mathrm{Tr}(\boldsymbol{\Sigma})} \times 100\%$.

4. Bullet Point 4 Explanation: 80% Variance Retention Criterion

- **What It Means:** Identify how many dimensions can be dropped while retaining the dominant 80% of variance.
- **Why It Is Essential:** Enables low-dimensional risk modeling and computational efficiency.
- **How to Formulate:** Compute cumulative PEV and find the smallest k where cumulative variance reaches $\geq 80\%$.

5. Bullet Point 5 Explanation: Low-Rank Approximation & Frobenius Norm Error

- **What It Means:** Reconstruct the covariance matrix using only top-2 eigenvectors and measure the information lost.
- **Why It Is Essential:** Quantifies compression error according to the Eckart–Young–Mirsky theorem.
- **How to Formulate:** Evaluate $\|\boldsymbol{\Sigma} - \boldsymbol{\Sigma}_2\|_F = \sqrt{\sum_{i=3}^{4}\lambda_i^2}$.